

Summer Training Report
on
Fraud Detection in Financial Transactions

Submitted in partial fulfilment of the requirements for the award of the
degree of

Bachelor of Computer Applications (BCA)
Guru Gobind Singh Indraprastha University, Delhi

Submitted To:

Dr. Charul Nigam
Associate Professor

Submitted By:

Aviral Ahuja
10121102023
BCA V E2



Institute of Information Technology &
Management, New Delhi – 1100 58

Batch (2023-2026)

CERTIFICATE

I, Aviral Ahuja (10121102023) certify that the Summer Training Project Report (BCA-331) entitled “Fraud Detection in Financial Transactions” is done by me and my group it is an authentic work carried out by me at Institute of Information Technology and Management. The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Name of the Student: Aviral Ahuja

Signature of the Student:

Certified that the Project Report (BCA-331) entitled “Fraud Detection in Financial Transactions” done by the above student is completed under my guidance.

Mohammad Jari
(External Guide)

Dr. Charul Nigam
(Internal Guide)

Mr. Asish Nayyar
(Training Coordinator)

Dr. Ganesh Wadhwani
(Head of the Department)

ACKNOWLEDGEMENT

I want to express my sincere gratitude to everyone who helped to finish this data science and machine learning project. I want to start by expressing my gratitude to Dr. Charul Nigam for all his help and support during this educational journey. His extensive experience and profound understanding have greatly influenced the course and result of this project.

I would like to extend my sincere gratitude to Mr. Ganesh Wadhwani for providing me with an outstanding training program that enabled an all-encompassing educational experience. My ability to successfully apply theoretical information to real-world situations was greatly aided by the tools and learning environment that were made available.

I am grateful to my peers who have offered their support and encouragement, contributing to a collaborative and stimulating learning environment. The exchange of ideas and constructive discussions have greatly enriched my project.

Thank you all for your invaluable contributions to this project.

Aviral Ahuja

SYNOPSIS

This project report is prepared as part of the partial fulfilment of the three-year UG curriculum at IITM, Janakpuri. The research aims to analyze credit card transaction data and detect fraudulent activities using machine learning techniques. With the increasing adoption of digital transactions, fraud detection has become a vital component in ensuring the safety and integrity of financial systems.

Problem Statement:

The detection of fraudulent transactions in financial systems is a major challenge due to the highly imbalanced nature of transaction data and the evolving strategies used by fraudsters. This project addresses the need to develop a reliable and automated fraud detection system that can accurately classify transactions as legitimate or fraudulent in real time.

Objectives:

The objective of this project is to explore, clean, and preprocess the PaySim dataset, build a classification model using Logistic Regression, evaluate its performance using appropriate metrics, and deploy it using a Streamlit web application. The project also aims to allow users to upload any dataset and select the column they want to predict, providing flexibility for binary classification problems.

Methodology:

The PaySim dataset, simulating mobile money transactions, is cleaned by removing unnecessary columns and applying one-hot encoding to categorical features. Exploratory data analysis is performed to understand patterns in the data. A logistic regression model is then trained on a subset of the dataset (due to size limitations) without sampling, and evaluated using metrics like accuracy, precision, recall, F1-score, confusion matrix, and ROC curve. The entire pipeline is integrated into an interactive Streamlit web app that allows users to upload their own dataset and predict fraud or other binary classes.

Expected Outcomes:

The project is expected to produce a streamlined and efficient fraud detection model that can be integrated into a web-based interface. It will help financial institutions quickly assess the legitimacy of transactions. The use of real-world synthetic data from the PaySim dataset ensures that the project is grounded in realistic financial behavior.

Limitations:

Due to the dataset's size, only the first few thousand records are processed to ensure the app performs efficiently on Streamlit. The current implementation supports only binary classification and may not perform well for multi-class scenarios or in cases where the data is highly unbalanced and no sampling is applied. Future improvements may include real-time data integration, advanced ensemble models, and support for multi-class predictions.

LIST OF THE FIGURES

S.No	Name	Page No.
1.	Fig 1.1.1 Example of storing a value in variable	1
2.	Fig 1.1.2 Identifying the data type of a variable	1
3.	Fig 1.1.3 Applying operators on variables	1
4.	Fig 1.1.4 Using conditional statement(if-else)	2
5.	Fig 1.1.5 Using ‘for’ loops	2
6.	Fig 1.1.6 Creating a function	2
7.	Fig 1.1.7 Handling error using ‘try’ and ‘except’	3
8.	Fig 1.2.1 Created a list	3
9.	Fig 1.2.2 Created a tuple	3
10.	Fig 1.2.3 Created a Set	4
11.	Fig 1.2.4 Created a dictionary	4
12.	Fig 1.2.5 Using loops for collections	4
13.	Fig 1.3.1 Created a markdown cell using ‘m’	5
14.	Fig 1.3.2 Created a code cell using ‘y’	5
15.	Fig 1.4.1 Created a array and calculated its mean	5
16.	Fig 1.4.2 Performed indexing and filtering on array	6
17.	Fig 2.1.1 Created a Series and a DataFrame	6
18.	Fig 2.1.2 Example of using a pandas function to read a csv file	7
19.	Fig 2.2.1 Handling missing values using ‘fillna()’	7
20.	Fig 2.2.2 Renaming a feature/column name	8
21.	Fig 2.2.3 Sorting a csv file on the basis of cloumns	8

22.	Fig 2.3.1 Importing matplotlib and seaborn libraries	9
23.	Fig 2.3.2 Created a bar chart	9
24.	Fig 2.3.3 Created a histogram	10
25.	Fig 2.3.4 Created a pie chart	10
26.	Fig 1.1.1 Created a scatter plot	11
27.	Fig 3.1.1 Creating a DataFrame and applying all functions of descriptive statistics	13
28.	Fig 3.2.1 Created a histogram to see distribution of data	13
29.	Fig 3.3.1 Example of using random library for probability	14
30.	Fig 4.1.1 Built a model using Linear Regression	15
31.	Fig 4.2.1 Built a model using Logistic Regression	16
32.	Fig 5.1.1 Example of using scaling on a model	18
33.	Fig 5.2.1 Using clustering on a model	19
34.	Fig 5.3.1 Using PCA on a dataset	20
35.	Fig 5.4.1 Identifying all outliers in the data	21
36.	Fig 6.2.1 Importing all libraries for project and setting up UI configuration	22
37.	Fig 6.2.2 Collecting data for the project	23
38.	Fig 6.2.3 Cleaning data for project	23
39.	Fig 6.2.4 Identifying features and target and dividing data for testing and training	23
40.	Fig 6.2.5 Training model through Logistic Regression	24
41.	Fig 6.2.6 Evaluating model on the basis of metrics	24
42.	Fig 6.2.7 Visualizing model through confusion matrix	24
43.	Fig 6.2.8 Visualizing model through ROC curve and	25

	giving its accuracy_score	
44.	Fig 6.3.1 Streamlit app website screenshot 1	26
45.	Fig 6.3.2 Streamlit app website screenshot 2	26
46.	Fig 6.3.3 Streamlit app website screenshot 3	27
47.	Fig 6.3.4 Streamlit app website screenshot 4	27

LIST OF THE TABLES

S.No	Name	Page No.
1.	Table 3.1.1 Function which come under descriptive statistics	12
2.	Table 4.3.1 Metrics used for Regression and Classification	16

INDEX

S.NO	TOPIC	PAGE NO
1.	Certificate	I
2.	Acknowledgement	II
3.	Synopsis	III
4.	List of figures	IV
5.	List of Tables	VII
5.	Chapter-1: Python Programming for Data Analysis	1
6.	1.1 Introduction to Python	1
7.	1.2 Working with Collections	3
8.	1.3 Using Jupyter Notebooks	5
9.	1.4 NumPy- Basics of Numerical Computation	5
10.	Chapter-2: Data Handling and Visualization	6
11.	2.1 Introduction to Pandas	6
12.	2.2 Data Cleaning	7
13.	2.3 Data Visualization	9
14.	Chapter-3: Statistics and Data Analysis Concepts	12
15.	3.1 Descriptive Statistics	12
16.	3.2 Data Distribution	13
17.	3.3 Introduction to Probability	14
18.	3.4 Hypothesis Testing	14
19.	Chapter-4: Introduction to Machine Learning- Regression and Classification	15

20.	4.1 What is Machine Learning?	15
21.	4.2 Linear Regression (Prediction)	16
22.	4.3 Logistic Regression (Classification)	16
23.	4.4 Model Improvement Basics	17
24.	Chapter-5: Clustering and Feature Engineering	18
25.	5.1 Feature Engineering Basics	18
26.	5.2 Clustering with k-Means	19
27.	5.3 PCA- Simplifying Data	20
28.	5.4 Outlier Detection	21
29.	Chapter-6: Capstone Project and Deployment Basics	22
30.	6.1 Project Planning	22
31.	6.2 Implementation Steps	22
32.	6.3 Introduction to Deployment	26
33.	6.4 Presentation and Review	27

1. Python Programming for Data Analysis

1.1 Introduction to Python

- **Variables:** Containers for storing data.

```
[3]: name = "Aviral"
    age = 21
    score = 89.5
    print(name)
    print(age)
    print(score)

Aviral
21
89.5
```

Fig 1.1.1 Example of storing a value in variable

- **Data Types:** int, float, str, bool

```
4]: type(score)

4]: float
```

Fig 1.1.2 Identifying the data type of a variable

- **Operators:** Arithmetic (+, -), Comparison (==, >), Logical (and, or).

```
a = 5
b = 3
print(a + b)

8
```

Fig 1.1.3 Applying operators on variables

- **Conditional Statements:** To make decisions.

```
if score > 50:  
    print("Pass")  
else:  
    print("Fail")  
  
Pass
```

Fig 1.1.4 Using conditional statement(if-else)

- **Loops:** Repeat tasks.

```
7]: for i in range(5):  
    print(i)  
  
0  
1  
2  
3  
4
```

Fig 1.1.5 Using 'for' loops

- **Functions:** Reusable blocks of code.

```
[9]: def greet(name):  
    return f"Hello, {name}!"  
    greet("Aviral")  
  
[9]: 'Hello, Aviral!'
```

Fig 1.1.6 Creating a function

- **Error Handling:** Prevent crashes.

```
[10]: try:
      print(10 / 0)
      except ZeroDivisionError:
          print("Can't divide by zero")

Can't divide by zero
```

Fig 1.1.7 Handling error using 'try' and 'except'

1.2 Working with Collections

Types of Collections:

- List: Ordered, mutable.

```
students = ["Avi", "Garv"]
students.append("Rithik")
students

['Avi', 'Garv', 'Rithik']
```

Fig 1.2.1 Created a list

- Tuple: Ordered, immutable.

```
[14]: grades = (85, 90, 78)
      print(grades)
      type(grades)

      (85, 90, 78)
[14]: tuple
```

Fig 1.2.2 Created a tuple

- Set: Unordered, unique values.

```
[15]: subjects = {"Math", "Science", "English"}  
      print(subjects)  
      type(subjects)  
  
      {'English', 'Math', 'Science'}  
[15]: set
```

Fig 1.2.3 Created a Set

- Dictionary: Key-value pairs.

```
[16]: marks = {"Avi": 80, "Rithik": 92}  
      print(marks)  
  
      {'Avi': 80, 'Rithik': 92}
```

Fig 1.2.4 Created a dictionary

Loops with Collections:

```
[17]: for name, mark in marks.items():  
      print(name, mark)  
  
      Avi 80  
      Rithik 92
```

Fig 1.2.5 Using loops for collections

1.3 Using Jupyter Notebooks

- **Key Features:**

- Markdown support (for headings, bullet points)
- Output below code cells
- Supports inline charts (e.g. Matplotlib)

Markdown Cell

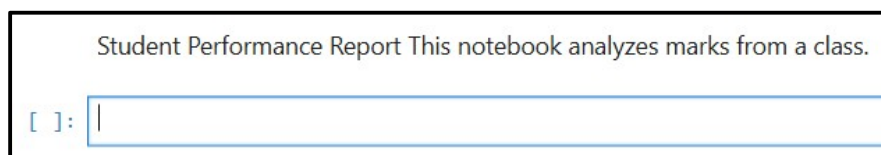


Fig 1.3.1 Created a markdown cell using ‘m’

Code Cell

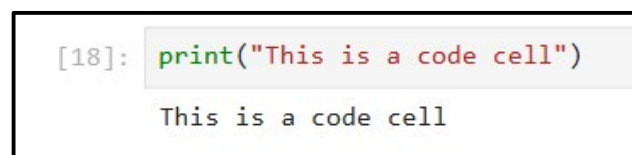


Fig 1.3.2 Created a code cell using ‘y’

1.4 NumPy – Basics of Numerical Computation

- Arrays (like lists, but faster and support math)

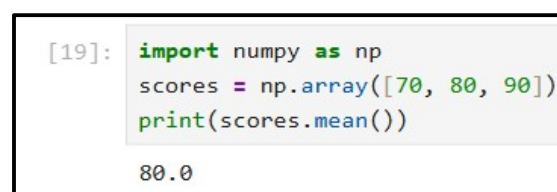


Fig 1.4.1 Created a array and calculated its mean

- Indexing & Filtering

```
[20]: print(scores[1])
      print(scores[scores > 75])

      80
      [80 90]
```

Fig 1.4.2 Performed indexing and filtering on array

2. Data Handling and Visualization

2.1 Introduction to Pandas

Key Concepts:

- DataFrame: 2D table (rows + columns)
- Series: 1D column

```
import pandas as pd
s=pd.Series([1,2,3,4,5], index=['a','b','c','d','e'])
print(s)

a    1
b    2
c    3
d    4
e    5
dtype: int64

pd.DataFrame({'Name':['Aviral','Parth','Manjot'],
              'Salary':[90000,80000,60000]})
```

	Name	Salary
0	Aviral	90000
1	Parth	80000
2	Manjot	60000

Fig 2.1.1 Created a Series and a DataFrame

```

movies=pd.read_csv("movies.csv")
print(movies)

```

	index	budget	genres
0	0	237000000	Action Adventure Fantasy Science Fiction
1	1	300000000	Adventure Fantasy Action
2	2	245000000	Action Adventure Crime
3	3	250000000	Action Crime Drama Thriller
4	4	260000000	Action Adventure Science Fiction
...	...	...	...
4798	4798	220000	Action Crime Thriller
4799	4799	9000	Comedy Romance
4800	4800	0	Comedy Drama Romance TV Movie
4801	4801	0	NaN
4802	4802	0	Documentary

	homepage	id
0	http://www.avatarmovie.com/	19995
1	http://disney.go.com/disneypictures/pirates/	285
2	http://www.sonypictures.com/movies/spectre/	206647
3	http://www.thedarkknighttrises.com/	49026

Fig 2.1.2 Example of using a pandas function to read a csv file

2.2 Data Cleaning

- Handling Missing Data:

```

employees.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
---  -
0   First Name          1000 non-null   object
1   Gender              855 non-null    object
2   Start Date          1000 non-null   object
3   Last Login Time     1000 non-null   object
4   Salary              1000 non-null   int64
5   Bonus %             1000 non-null   float64
6   Senior Management   933 non-null    object
7   Team                957 non-null    object
dtypes: float64(1), int64(1), object(6)
memory usage: 62.6+ KB

employees.fillna("unknown",inplace=True)

employees.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
---  -
0   First Name          1000 non-null   object
1   Gender              1000 non-null   object
2   Start Date          1000 non-null   object
3   Last Login Time     1000 non-null   object
4   Salary              1000 non-null   int64
5   Bonus %             1000 non-null   float64
6   Senior Management   1000 non-null   object
7   Team                1000 non-null   object
dtypes: float64(1), int64(1), object(6)
memory usage: 62.6+ KB

```

Fig 2.2.1 Handling missing values using 'fillna()'

- Formatting & Renaming:

```
movies.rename(columns={"keywords":"Keynotes"},inplace=True)
```

	index	budget	genres	homepage	id	Keynotes	original_language	original_title	overview	popularity
0	0	237000000	Action Adventure Fantasy Science Fiction	http://www.avatarmovie.com/	19995	culture clash future space war space colony so...	en	Avatar	In the 22nd century, a paraplegic Marine is di...	150.43757
1	1	300000000	Adventure Fantasy Action	http://disney.go.com/disneypictures/pirates/	285	ocean drug abuse exotic island east india trad...	en	Pirates of the Caribbean: At World's End	Captain Barbossa, long believed to be dead, ha...	139.08261
2	2	245000000	Action		385647	spy based on novel			A cryptic message from	107.37778

Fig 2.2.2 Renaming a feature/column name

- Sorting & Grouping:

```
movies[['original_title','release_date']].head()
```

	original_title	release_date
0	Avatar	2009-12-10
1	Pirates of the Caribbean: At World's End	2007-05-19
2	Spectre	2015-10-26
3	The Dark Knight Rises	2012-07-16
4	John Carter	2012-03-07

Fig 2.2.3 Sorting a csv file on the basis of cloumns

2.3 Data Visualization

Purpose: Understand data through graphs and plots.

- Matplotlib & Seaborn:

```
[*]: import matplotlib.pyplot as plt  
import seaborn as sns
```

Fig 2.3.1 Importing matplotlib and seaborn libraries

Types of Plots:

- Bar Chart:

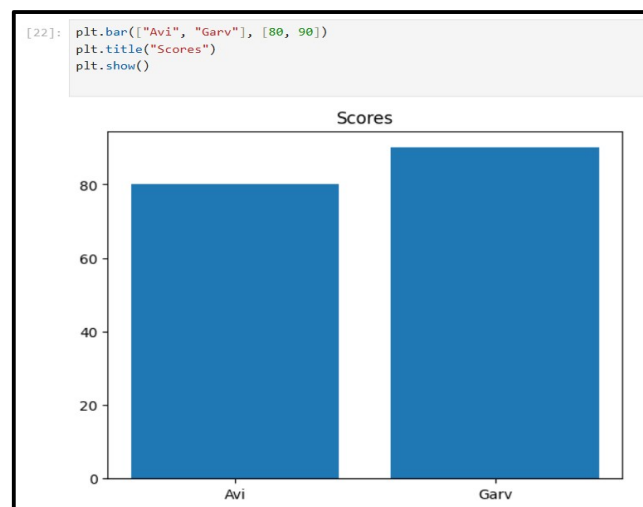


Fig 2.3.2 Created a bar chart

- Histogram:

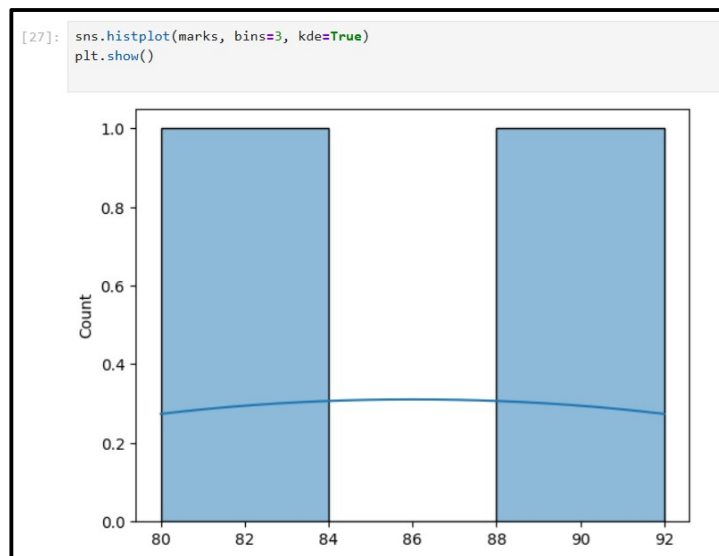


Fig 2.3.3 Created a histogram

- Pie Chart:

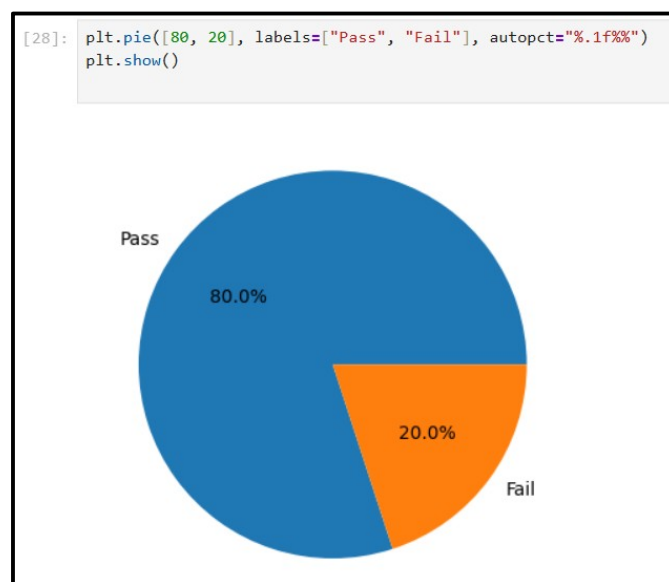


Fig 2.3.4 Created a pie chart

- Scatter Plot:

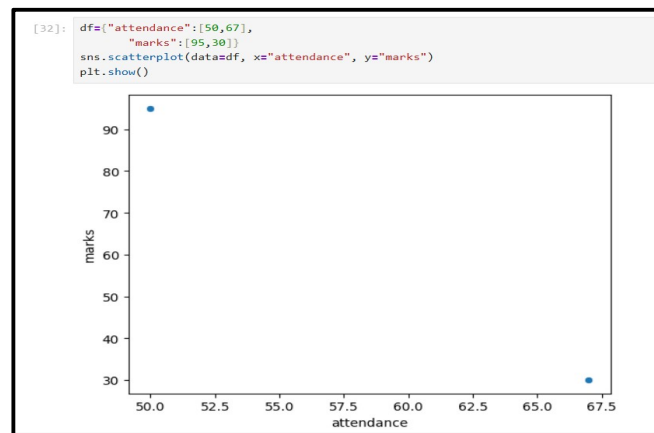


Fig 1.1.1 Created a scatter plot

3. Statistics and Data Analysis Concepts

Statistics helps us:

- Understand data trends and variation
- Make informed decisions (e.g., whether one class performed better than another)
- Test hypotheses using logic, not guesses
- Prepare data for machine learning

3.1 Descriptive Statistics

Descriptive statistics describe the central tendency, spread, and shape of a dataset.

Table 3.1.1 Function which come under descriptive statistics

Term	Meaning	Python Example
Mean	Average value	<code>df["Marks"].mean()</code>
Median	Middle value	<code>df["Marks"].median()</code>
Mode	Most frequent	<code>df["Marks"].mode()</code>
Range	Max - Min	<code>df["Marks"].max() - df["Marks"].min()</code>
Standard Deviation	Spread from mean	<code>df["Marks"].std()</code>

```
[33]: import pandas as pd

df = pd.DataFrame({
    "Student": ["Avi", "Garv", "Rithik", "Krish", "Shubh"],
    "Marks": [75, 80, 85, 90, 95]
})

print("Mean:", df["Marks"].mean())
print("Median:", df["Marks"].median())
print("Mode:", df["Marks"].mode()[0])
print("Range:", df["Marks"].max() - df["Marks"].min())
print("Standard Deviation:", df["Marks"].std())

Mean: 85.0
Median: 85.0
Mode: 75
Range: 20
Standard Deviation: 7.905694150420948
```

Fig 3.1.1 Creating a DataFrame and applying all functions of descriptive statistics

3.2 Data Distributions

A distribution shows how values in a dataset are spread.

- Normal distribution = Bell curve → most values around the mean
- Skewed distribution = shifted left or right

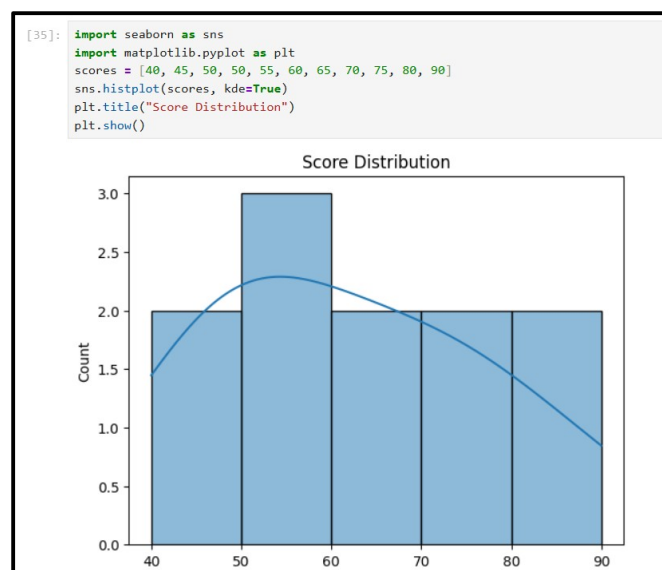


Fig 3.2.1 Created a histogram to see distribution of data

3.3 Introduction to Probability

Probability measures the likelihood of an event.

- Range: 0 to 1
- Common tools: cards, dice, coin flips

```
[36]: red_cards = 26
      total_cards = 52
      print("Probability of red card:", red_cards / total_cards)

      Probability of red card: 0.5

[37]: import random
      coin = random.choice(["Heads", "Tails"])
      print("You got:", coin)

      You got: Tails
```

Fig 3.3.1 Example of using random library for probability

3.4 Hypothesis Testing

Steps:

- Null Hypothesis (H_0): No difference (e.g., “Class A and Class B perform the same”)
- Alternative Hypothesis (H_1): There is a difference
- Choose test: t-test, chi-square, etc.
- Calculate p-value
- If $p < 0.05$, reject H_0 (significant difference exists)

4. Introduction to Machine Learning

What is Machine Learning?

Machine Learning (ML) is a method where computers learn from data and make predictions or decisions without being explicitly programmed.

ML Workflow:

- Data Collection
- Data Cleaning & Preprocessing
- Split Data (Train/Test)
- Build Model
- Evaluate Model
- Use Model for Prediction

4.1 Linear Regression (Prediction)

Linear Regression is used to predict a numeric value (e.g., marks, sales, temperature) based on one or more input features.

FORMULA

$$Y=mx+c$$

```
[38]: import pandas as pd
      from sklearn.linear_model import LinearRegression
      from sklearn.model_selection import train_test_split
      from sklearn.metrics import mean_absolute_error, mean_squared_error
      data = pd.DataFrame({
          "Hours_Studied": [1, 2, 3, 4, 5, 6],
          "Marks": [50, 55, 60, 65, 70, 75]
      })
      X = data[["Hours_Studied"]]
      y = data[["Marks"]]
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
      model = LinearRegression()
      model.fit(X_train, y_train)
      predictions = model.predict(X_test)
      print("MAE:", mean_absolute_error(y_test, predictions))
      print("MSE:", mean_squared_error(y_test, predictions))

      MAE: 3.552713678800501e-15
      MSE: 2.524354896707238e-29
```

Fig 4.1.1 Built a model using Linear Regression

4.2 Logistic Regression (Classification)

Logistic Regression is used for binary classification (e.g., Pass/Fail, Fraud/Not Fraud).

```
[40]: from sklearn.linear_model import LogisticRegression
      from sklearn.metrics import accuracy_score, confusion_matrix
      df = pd.DataFrame({
          "Internal_Marks": [10, 20, 30, 40, 50, 60],
          "Attendance": [60, 70, 75, 80, 85, 90],
          "Result": [0, 0, 0, 1, 1, 1]
      })
      X = df[["Internal_Marks", "Attendance"]]
      y = df["Result"]
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
      log_model = LogisticRegression()
      log_model.fit(X_train, y_train)
      pred = log_model.predict(X_test)
      print("Accuracy:", accuracy_score(y_test, pred))
      print("Confusion Matrix:\n", confusion_matrix(y_test, pred))

Accuracy: 1.0
Confusion Matrix:
[[1 0]
 [0 1]]
```

Fig 4.2.1 Built a model using Logistic Regression

4.3 Model Evaluation Metrics

Table 4.3.1 Metrics used for Regression and Classification

Type	Metric	Purpose
Regression	MAE / MSE	How close predicted numbers are
Classification	Accuracy	% correctly classified
Classification	Confusion Matrix	TP, FP, FN, TN
Classification	Precision/Recall/F1	Deep error analysis

4.4 Model Improvement Basics

Once you build a basic machine learning model (like linear or logistic regression), the next step is to **make it better**.

Improving a model means:

- Increasing **accuracy** (for classification)
- Decreasing **error** (for regression)
- Ensuring it works well **on new data**, not just training data

A. Using More Features (Feature Engineering)

- More relevant features = better model understanding.

B. Avoiding Overfitting

- Overfitting means your model performs very well on training data, but fails on unseen test data. Think of it like memorizing answers instead of understanding the topic.

5. Clustering and Feature Engineering

5.1 Feature Engineering Basics

Feature engineering is the process of creating and transforming data columns to improve machine learning model performance.

Why it's important:

- Raw data isn't always usable.
- Better features = better predictions.
- Helps models understand numbers, categories, or text.

Techniques:

Scaling and Normalization

- Ensures values are on the same scale.
- Especially useful for models that rely on distance (e.g., k-NN, k-Means).

```
[41]: from sklearn.preprocessing import MinMaxScaler, StandardScaler

      scaler = MinMaxScaler()
      df_scaled = scaler.fit_transform(df)

[42]: import pandas as pd

      df = pd.DataFrame({
          "Gender": ["Male", "Female", "Female", "Male"]
      })

      df_encoded = pd.get_dummies(df, drop_first=True)
      print(df_encoded)
```

	Gender_Male
0	True
1	False
2	False
3	True

Fig 5.1.1 Example of using scaling on a model

5.2 Clustering with k-Means

Clustering is an unsupervised learning method where you group data based on similarity.

- You don't need labeled output.
- Groups are called clusters.

Use Case:

- Grouping customers based on purchase behavior
- Grouping students based on performance patterns

How k-Means Works:

1. Pick k number of clusters
2. Randomly assign cluster centers (centroids)
3. Assign data points to nearest centroid
4. Recalculate centroids
5. Repeat until stable

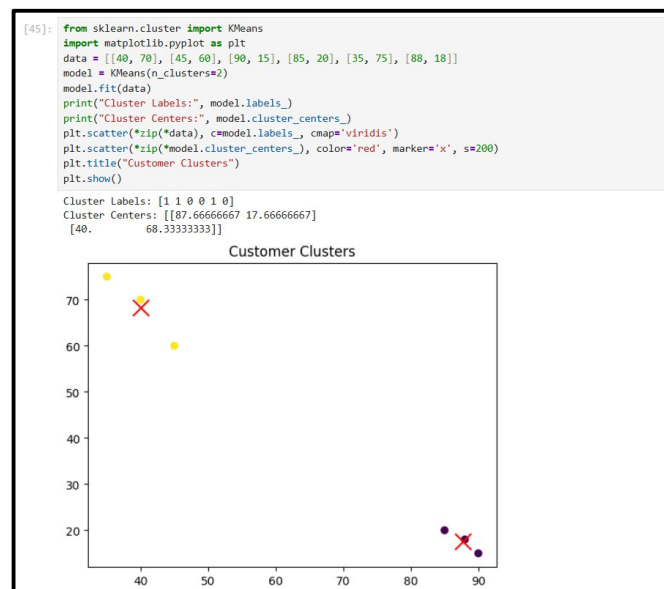


Fig 5.2.1 Using clustering on a model

5.3 PCA – Principal Component Analysis

PCA (Principal Component Analysis) is a dimensionality reduction technique.

- Reduces number of columns (features) while preserving maximum variance
- Makes visualization easier in 2D or 3D
- Helps speed up training

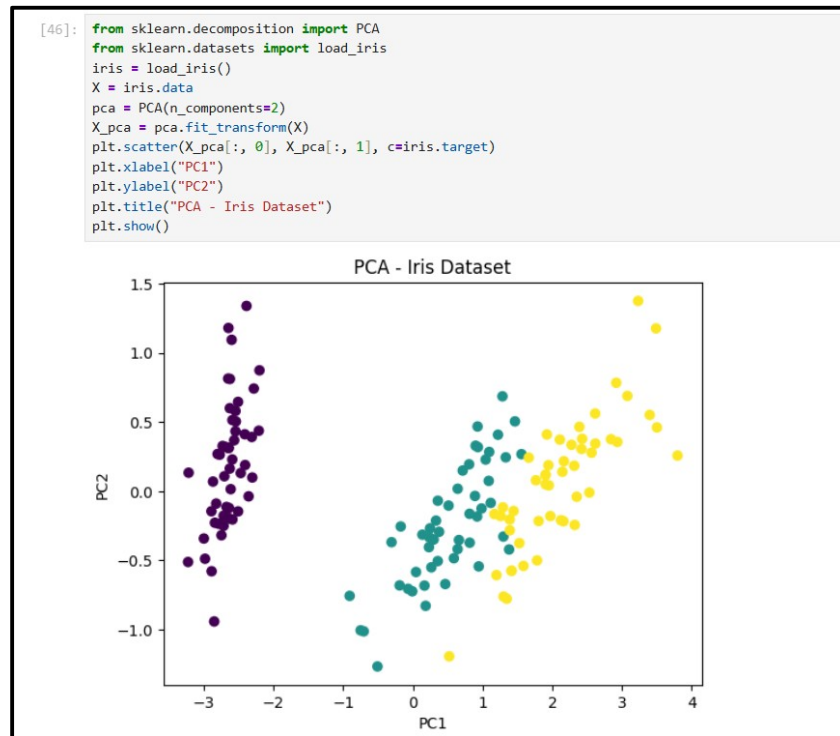


Fig 5.3.1 Using PCA on a dataset

5.4 Outlier Detection

An outlier is a value that is significantly higher or lower than the rest of the data.

- Outliers can distort model performance.
- Detecting them improves model quality.

```
[47]: import numpy as np
data = [10, 12, 14, 15, 16, 18, 100]
q1 = np.percentile(data, 25)
q3 = np.percentile(data, 75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = [x for x in data if x < lower or x > upper]
print("Outliers:", outliers)

Outliers: [100]
```

Fig 5.4.1 Identifying all outliers in the data

6. Capstone Project – Fraud Detection

Project Title:

"Fraud Detection in Financial Transactions using Logistic Regression and Streamlit"

Objective:

Build a machine learning model to detect fraudulent financial transactions using the PaySim dataset and deploy it via a web interface.

6.1 Project Planning

STEP	DESCRIPTION
Problem	Classify transactions as fraudulent or not
Input Features	Amount, old/new balances, transaction type
Output	Fraudulent (1) or Not Fraudulent (0)
Goal	Minimize false negatives (undetected frauds)

6.2 Implementation Workflow

1. Project Setup & UI Configuration

```
1  import streamlit as st
2  import pandas as pd
3  import numpy as np
4  from sklearn.model_selection import train_test_split
5  from sklearn.linear_model import LogisticRegression
6  from sklearn.metrics import (
7      classification_report, confusion_matrix, roc_curve,
8      auc, accuracy_score
9  )
10 import matplotlib.pyplot as plt
11 import seaborn as sns
12 import io
13
14 # Page config
15 st.set_page_config(page_title="Fraud Detection App", layout="centered")
16
17 st.markdown("<h1 style='color:#3B82F6;'>🔒 Fraud Detection Dashboard</h1>", unsafe_allow_html=True)
18 st.markdown("This app uses Logistic Regression to predict fraudulent transactions from the PaySim dataset.")
```

Fig 6.2.1 Importing all libraries for project and setting up UI configuration

2. Data Collection

```
file = st.file_uploader("📁 Upload your PaySim-like CSV file", type=["csv"])

if file:
    df = pd.read_csv(file)

    st.subheader("📄 Raw Dataset Preview")
    st.dataframe(df.head())
```

Fig 6.2.2 Collecting data for the project

3. Data Validation & Cleaning

```
if file:
    df = pd.read_csv(file)

    st.subheader("📄 Raw Dataset Preview")
    st.dataframe(df.head())

    if "isFraud" not in df.columns:
        st.error("❌ The dataset must contain a column named 'isFraud'. Please upload a valid PaySim dataset.")
        st.stop()

    # Drop unnecessary columns
    drop_cols = ['nameOrig', 'nameDest', 'isFlaggedFraud']
    df = df.drop(columns=[col for col in drop_cols if col in df.columns])

    # Encode transaction type
    if "type" in df.columns:
        df = pd.get_dummies(df, columns=["type"], drop_first=True)
```

Fig 6.2.3 Cleaning data for project

4. Feature Selection & Train-Test Split

```
# Feature/Target split
X = df.drop(columns=["isFraud"])
y = df["isFraud"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Fig 6.2.4 Identifying features and target and dividing data for testing and training

5. Model Training

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[: , 1]

st.success("✅ Logistic Regression model trained successfully!")
```

Fig 6.2.5 Training model through Logistic Regression

6. Model Evaluation

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
st.metric(label="📊 Model Accuracy", value=f"{accuracy*100:.2f}%")

# Classification Report
st.subheader("📊 Classification Report")
report_text = classification_report(y_test, y_pred)
st.text(report_text)

# Downloadable Report
report_dict = classification_report(y_test, y_pred, output_dict=True)
report_df = pd.DataFrame(report_dict).transpose()
report_csv = report_df.to_csv().encode()
st.download_button("📄 Download Report as CSV", data=report_csv, file_name='classification_report.csv', mime='text/csv')
```

Fig 6.2.6 Evaluating model on the basis of metrics

7. Confusion Matrix Visualization

```
st.subheader("📊 Confusion Matrix")
fig1, ax1 = plt.subplots()
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues', ax=ax1)
ax1.set_xlabel("Predicted")
ax1.set_ylabel("Actual")
st.pyplot(fig1)
```

Fig 6.2.7 Visualizing model through confusion matrix

8. ROC Curve + AUC Score

```
st.subheader("📊 ROC Curve")
fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)

fig2, ax2 = plt.subplots()
ax2.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}", color='green')
ax2.plot([0, 1], [0, 1], linestyle='--', color='gray')
ax2.set_xlabel("False Positive Rate")
ax2.set_ylabel("True Positive Rate")
ax2.set_title("ROC Curve")
ax2.legend()
st.pyplot(fig2)

st.markdown(f"AUC Score: {roc_auc:.2f} 📊 {'Excellent model!' if roc_auc > 0.8 else 'Consider tuning the model.'}")
```

Fig 6.2.8 Visualizing model through ROC curve and giving its accuracy_score

6.3 Introduction to Deployment

STREAMLIT APP

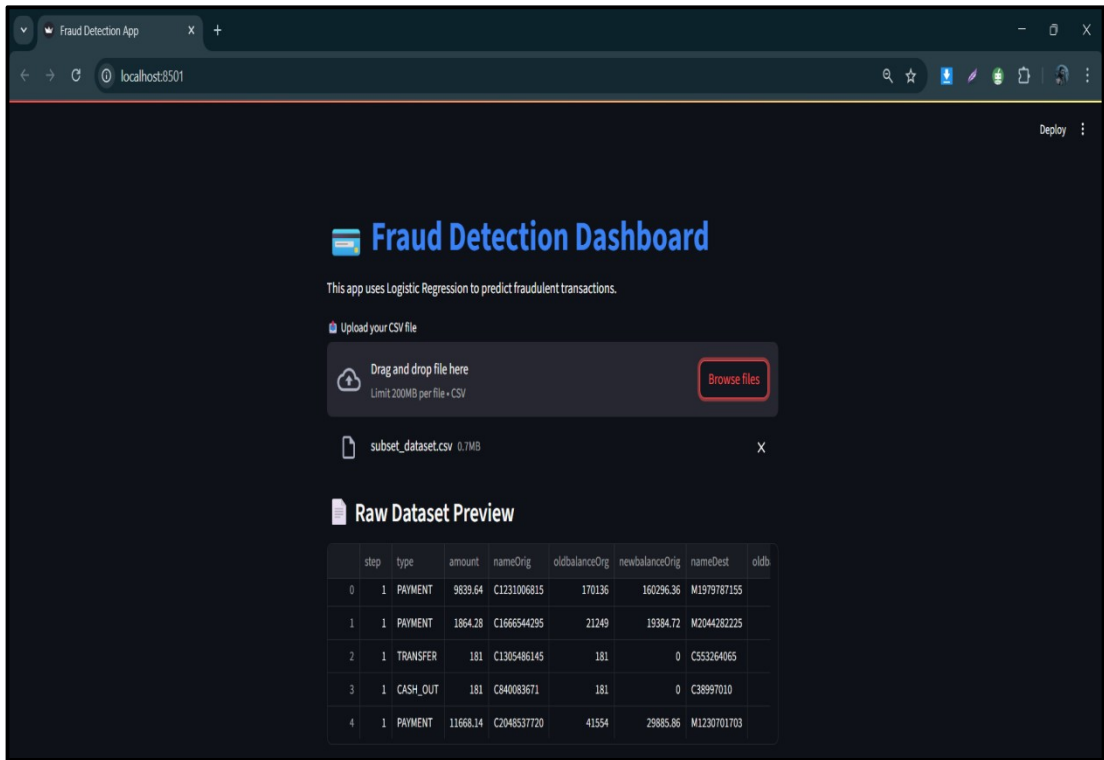


Fig 6.3.1 Streamlit app website screenshot 1

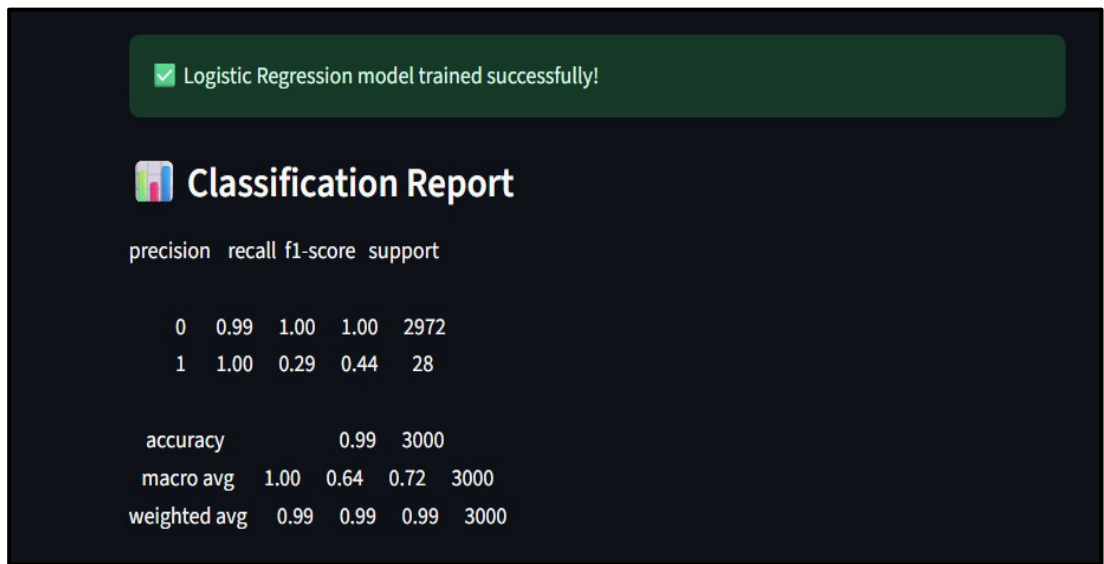


Fig 6.3.2 Streamlit app website screenshot 2

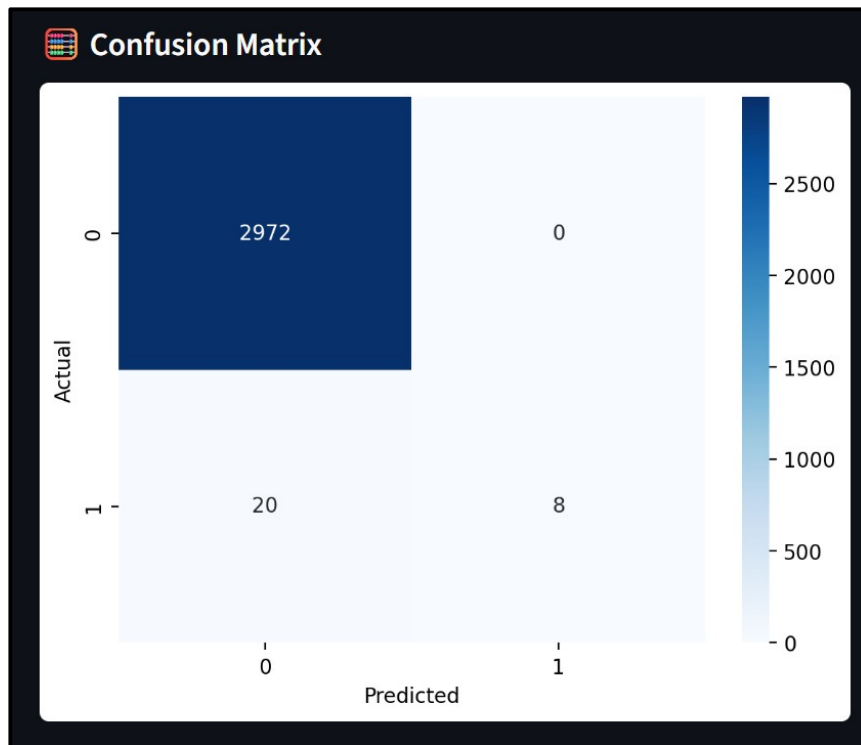


Fig 6.3.3 Streamlit app website screenshot 3

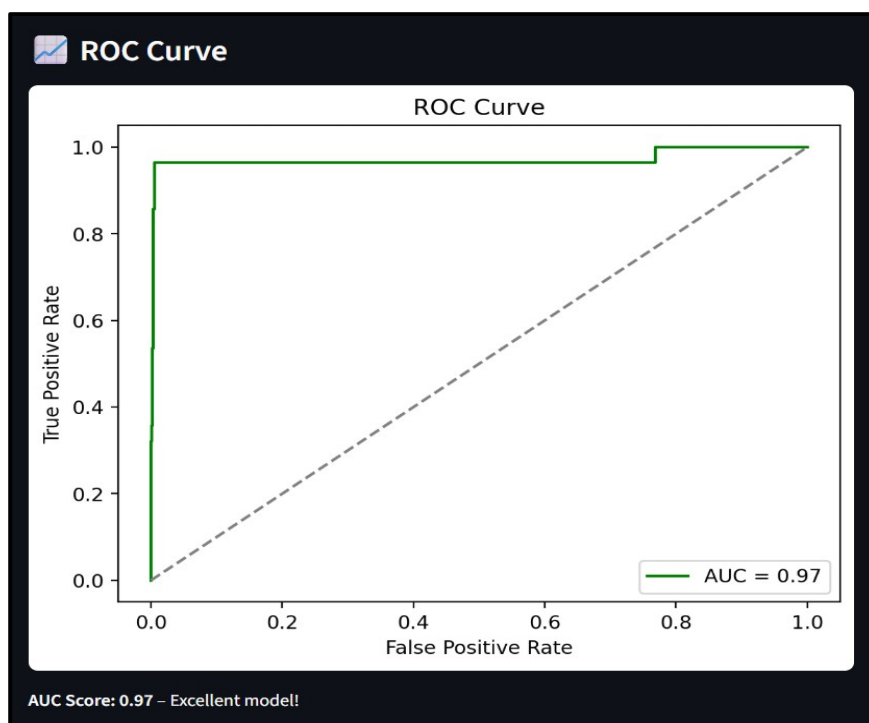


Fig 6.3.4 Streamlit app website screenshot 4